

Projeto 2 - Equações de ondas.

6 de abril de 2025

Universidade de São Paulo - Instituto de Física de São Carlos.

Docente: Francisco C. Alcaraz

Tony Maciel Alexander. # USP: 12624850

Tarefa 1

(I) Faça um programa que calcule as ondas perfeitas de velocidade c em um meio não dissipativo ou dispersivo de comprimento L . Especialize seu programa para a situação em que $Y(x, 0) = Y_0(x)$ é dada e $dY(x, t)/dt|_{t=0} = 0$ (onda parte do repouso). Considere fronteiras fixas. Desta forma os parâmetros do programa serão L , c , Δx , r e Y_0 (que pode ser fornecida numa subrotina com a mesma discretização). Para testes escolha $L = 1\text{m}$ e $c = 300\text{m/s}$. Considere um pacote Gaussiano inicial

$$Y(x, 0) = Y_0(x) = \exp[-(x - x_0)^2/\sigma^2] \quad (1)$$

com $x_0 = L/3$ e $\sigma = L/30$.

(Ia) Escolha Δx apropriado para uma boa aproximação da equação (0.2) (reveja o que aprendeu em física computacional I) e escolha $r = 1$. Mostre graficamente o perfil da onda obtida para vários tempos.

(Ia1) Que valor de Δx você usou?

(Ia2) O pacote se deforma?

(Ia3) Discuta as reflexões.

(Ia4) Discuta as interferências.

(Ia5) A configuração inicial será repetida quando?

(Ib) Use o Δx do item anterior mas escolha $r = 2$. Compare seus resultados com o item anterior. Explique seus resultados.

(Ic) Use o Δx do item (Ia) mas escolha $r = 0.25$. Compare seus resultados com o do item (Ia). Explique seus resultados.

Para resolver essa questão, decidi criar um "derived type" em fortran ¹ que armazena dados da onda, como sua velocidade, seu comprimento, a discretização do espaço e os três vetores que representam o deslocamento Y da onda ao longo do tempo em três iterações. Além disso, armazenei toda subrotina em um módulo separado chamado "wave_wfe.f90" (wave with fixed ends).

Iniciei a onda com um formato inicial (nessa tarefa foi um pulso gaussiano), depois iterei a simulação para um certo número de passos, salvando as posições para um certo múltiplo do número de iterações (controlado pelo "lag") e salvei tudo em um arquivo. O código fonte está nas figuras 1, 2 (módulo com as subrotinas) e 3 (programa principal).

¹"Derived types" são essencialmente "structs" em C, que por sua vez, servem para empacotar códigos, tornando programas mais enxutos (caso for necessário generalizá-lo) e fáceis de entender (na minha opinião).

```

10 module wave_wfe
11 use, intrinsic :: iso_fortran_env, only: sp=>real32 , dp=>real64
12 implicit none
13
14 type:: wave
15   real(dp)           :: dx, c, r
16   integer            :: L, npoints
17   real(dp), allocatable :: sys(:, :)
18
19   contains
20
21   procedure :: initialize, iterate, simulate
22 end type
23
24 contains
25
26 subroutine initialize(self, dx, r, c, L)
27   class(wave), intent(out) :: self
28   real(dp), intent(in)     :: dx, r, c
29   integer, intent(in)      :: L
30   real(dp)                 :: x_0 , sigma
31   integer                  :: i
32
33   self%dx      = dx
34   self%c       = c
35   self%r       = r
36   self%L       = L
37   self%npoints = int(L/dx)
38   allocate(self%sys(0:self%npoints, 3)) ! same wave at 3 different times.
39   ! INITIALIZING. MODIFY THIS...
40   x_0 = real(self%npoints, dp) / 3.0_dp
41   sigma = real(self%npoints, dp) / 30.0_dp
42   do i = 1, self%npoints-1
43     self%sys(i, :) = dexp(-((i - x_0) / sigma)**2)
44   end do
45   ! BOUNDARY CONDITIONS: FIXED ENDS
46   self%sys(0, :) = 0.0_dp
47   self%sys(self%npoints, :) = 0.0_dp
48 end subroutine initialize
49
50 ! finds next position of wave.
51 subroutine iterate(self)
52   class(wave), intent(in out) :: self
53   integer                    :: i
54
55   do i = 1, self%npoints-1
56     self%sys(i, 3) = 2.0_dp*(1.0_dp - self%r**2)*self%sys(i, 2) + &
57     (self%r**2)*(self%sys(i+1, 2) + self%sys(i-1, 2)) - self%sys(i, 1)
58   end do
59   ! BOUNDARY CONDITIONS: FIXED ENDS

```

Figura 1: Módulo com funções para simular uma onda unidimensional perfeita partindo do repouso com pulso inicial gaussiano (parte 1).

```

57 | | (self%r**2)*(self%sys(i+1,2) + self%sys(i-1,2)) - self%sys(i,1)
58 | end do
59 | ! BOUNDARY CONDITIONS: FIXED ENDS
60 | self%sys(0,3) = 0.0_dp
61 | self%sys(self%npoints,3) = 0.0_dp
62 |
63 | ! get ready for next iteration
64 | do i = 0, self%npoints
65 | | self%sys(i,1) = self%sys(i,2)
66 | | self%sys(i,2) = self%sys(i,3)
67 | end do
68 | end subroutine iterate
69 |
70 | ! simulates waves until time t, saving results to given file.
71 | subroutine simulate(self, t, lag, filename)
72 | class(wave), intent(in out) :: self
73 | integer, intent(in) :: t, lag ! number of iterations and time between saves.
74 | character(len=*), intent(in) :: filename ! file to overwrite data.
75 | real(dp), allocatable :: data(:, :) ! For column first, I have to save before write.
76 | integer :: i, j, io, idx
77 |
78 | allocate(data(0:self%npoints, int(t/lag)+2))
79 | ! save initial configuration
80 | do i = 0, self%npoints
81 | | data(i,1) = i*self%dx
82 | | data(i,2) = self%sys(i,2)
83 | end do
84 |
85 | idx = 3
86 | do i = 1, (t - mod(t,lag)) ! avoid unnecessary iterations
87 | | call self%iterate()
88 | | if (mod(i,lag) .eq. 0) then
89 | | | data(:,idx) = self%sys(:,2)
90 | | | idx = idx + 1
91 | | end if
92 | end do
93 |
94 | open(newunit=io, file=filename, action="write")
95 | do i = 0, self%npoints
96 | | write(io,*) (data(i,j), j = 1, (int(t/lag)+2)) ! "dynamic formatting" :o
97 | end do
98 | close(io)
99 |
100 | end subroutine simulate
101 |
102 | end module wave_wfe

```

Figura 2: Módulo com funções para simular uma onda unidimensional perfeita partindo do repouso com pulso inicial gaussiano (parte 2).

```

8 program tarefa1a
9 use wave_wfe
10 implicit none
11
12 type(wave) :: wave1
13
14 call wave1%initialize(dx=1e-3_dp, r=1.0_dp, c = 3e2_dp, L=1)
15 call wave1%simulate(t=2000, lag=200, filename="saida-1a-12624850.dat")
16 ! dt = rdx/c = 3.33e-6 s
17
18 end program tarefa1a

```

Figura 3: Programa principal que invoca as subrotinas das figuras 1 e 2.

Para o restante do relatório, considere $L = 1m$, $c = 300m/s$ e $\Delta x = 10^{-3}m$.

Nas figuras 4, 5 e 6, estão as posições dos pulsos da onda ao longo do tempo para $r = 1$, $r = 2$ e $r = 0.25$. Parece que só para $r = 1$ e $r = 0.25$ que a simulação converge para resultados coerentes. Para $r = 2$, o pulso diverge porque é como se a onda se propagasse com uma velocidade c tal que $c = 2dx/dt$, mas a cada iteração, a simulação só consegue fazer a onda propagar de dx em dx , que seria mais devagar que a verdadeira velocidade da onda, logo o código "fica pra trás" e os erros vão acumulando. O mesmo raciocínio pode ser usado para $r = 0.25$, mas dessa vez, o resultado não é tão ruim porque a velocidade da onda é menor que o máximo permitido da simulação, então o pior que pode acontecer é o programa querer antecipar a posição da onda e ter erros que acumulam. Quando $r = 1$, temos sincronia entre a velocidade da onda e a velocidade da simulação, então essa seria a melhor opção.

A configuração inicial será repetida após t iterações, onde t é dado pela equação 2. Ou seja, será repetida após $t(r = 1) = 2000$ iterações (aproximadamente $6.7ms$), $t(r = 2) = 1000$ iterações (aproximadamente $3.3ms$) e $t(r = 0.25) = 8000$ iterações (aproximadamente $26.7ms$).

$$t = \frac{2L/c}{dt} = \frac{2L}{rdx} \quad (2)$$

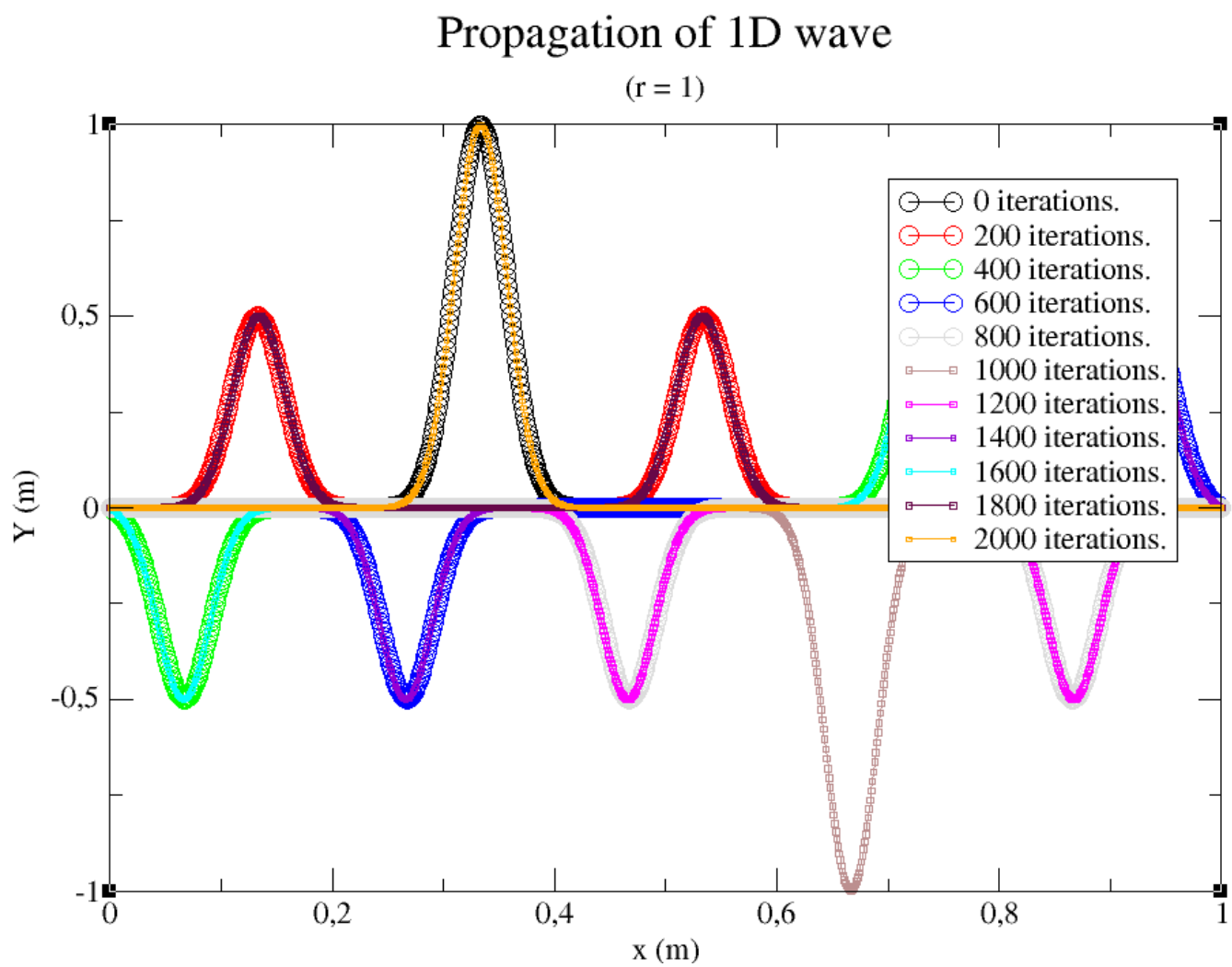


Figura 4: Deslocamento de uma onda unidimensional. A simulação começa na gaussiana preta (em cima).

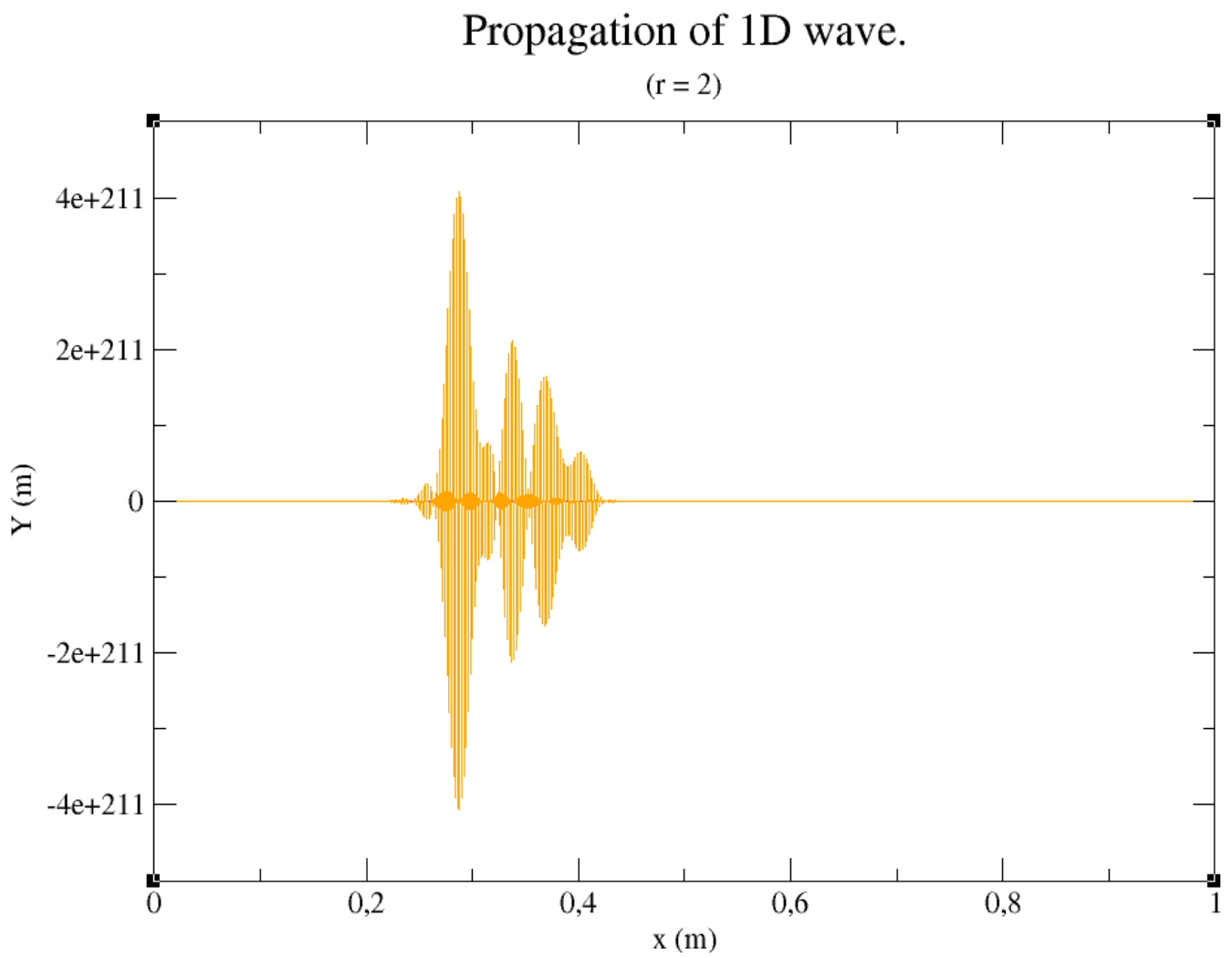


Figura 5: Deslocamento de uma onda unidimensional com $r = 2$. Resultados não coerentes.

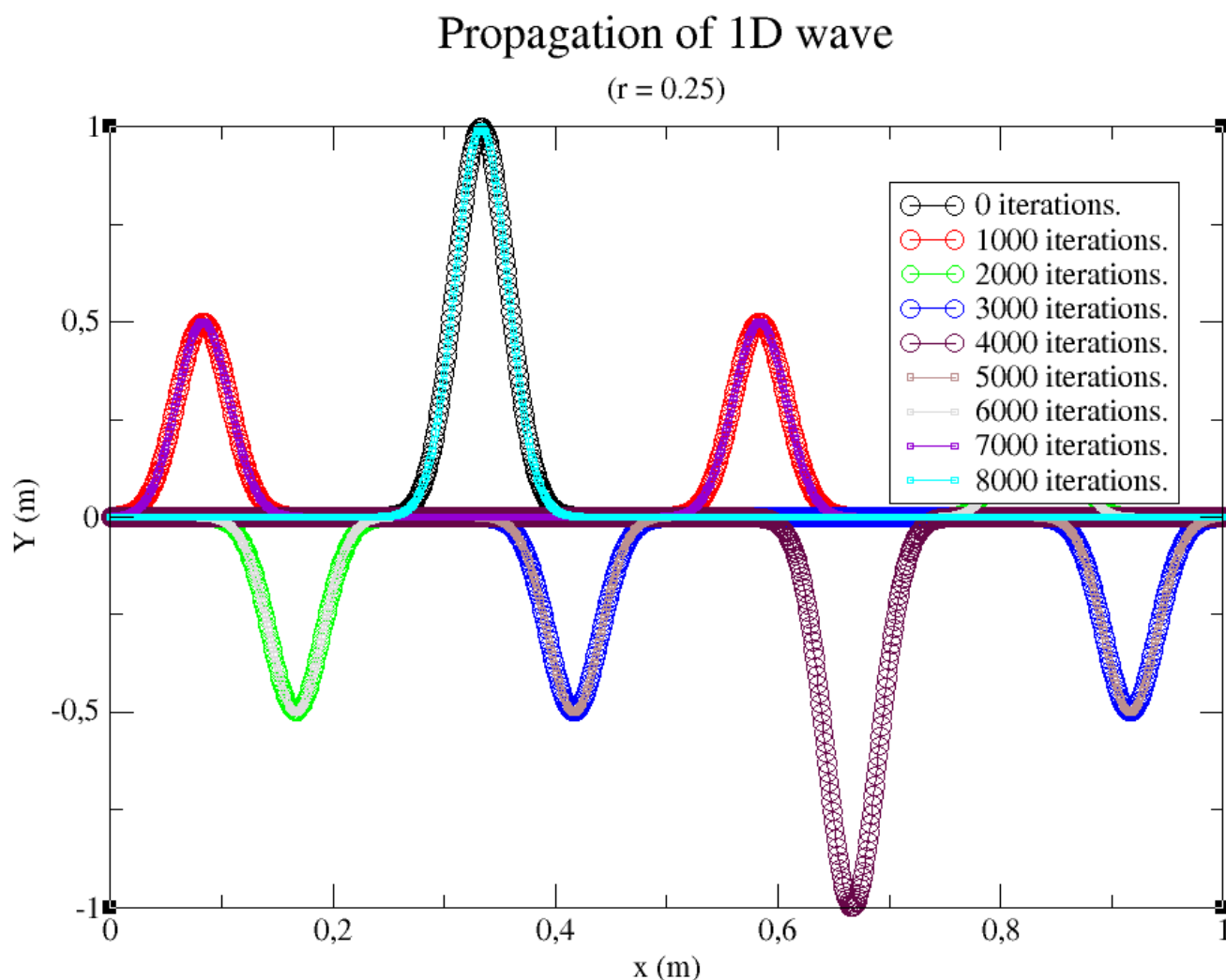


Figura 6: Deslocamento de uma onda unidimensional com $r = 0.25$. Resultados com um pouco de erro.

Nas figuras 4 e 6, dá pra ver que a onda começa se espalhando, atinge as fronteiras, volta com Y invertido, se junta embaixo, atinge as fronteiras de novo e se junta em cima para repetir o ciclo.

Tarefa 2

- (II) Considere o item (I) mas com $Y_0(x)$ dado com num violão, isto é:
 (imagine um violão...)
 Repita as questões (Ia2)-(Ia5).

O código fonte dessa tarefa é quase igual ao da tarefa anterior, a única diferença está mostrada na figura 7, onde modifiquei a subrotina "initialize" para ter um pulso de violão e não gaussiana. O resto é igual.


```

26 subroutine initialize(self, dx, r, c, L)
27   class(wave), intent(out) :: self
28   real(dp), intent(in)      :: dx, r, c
29   integer, intent(in)       :: L
30   real(dp)                  :: x_kink
31   integer                    :: i
32
33   self%dx = dx
34   self%c = c
35   self%r = r
36   self%L = L
37   self%npoints = int(L/dx)
38   allocate(self%sys(0:self%npoints, 3)) ! same wave at 3 different times.
39
40   ! INITIALIZING. MODIFY THIS...
41   x_kink = real(self%npoints,dp) / 4.0_dp
42   do i = 1, int(x_kink) ! y = x
43     self%sys(i,:) = self%dx*i
44   end do
45
46   do i = int(x_kink), self%npoints-1 ! y = -x/3 + L/3
47     self%sys(i,:) = -1.0_dp/3.0_dp*self%dx*i + 1.0_dp/3.0_dp*self%npoints*self%dx
48   end do
49   ! BOUNDARY CONDITIONS: FIXED ENDS
50   self%sys(0,:) = 0.0_dp
51   self%sys(self%npoints,:) = 0.0_dp
52 end subroutine initialize

```

Figura 7: única mudança no código fonte da tarefa 1, na figura 1.

A propagação dessa onda está mostrada na figura 8 . O comportamento é análogo ao pulso gaussiano: ele se propaga para ambos lados, inverte Y, se junta, inverte Y de novo e volta na configuração inicial após $t(r = 1) = 2000$ iterações.

Propagations of 1D wave.

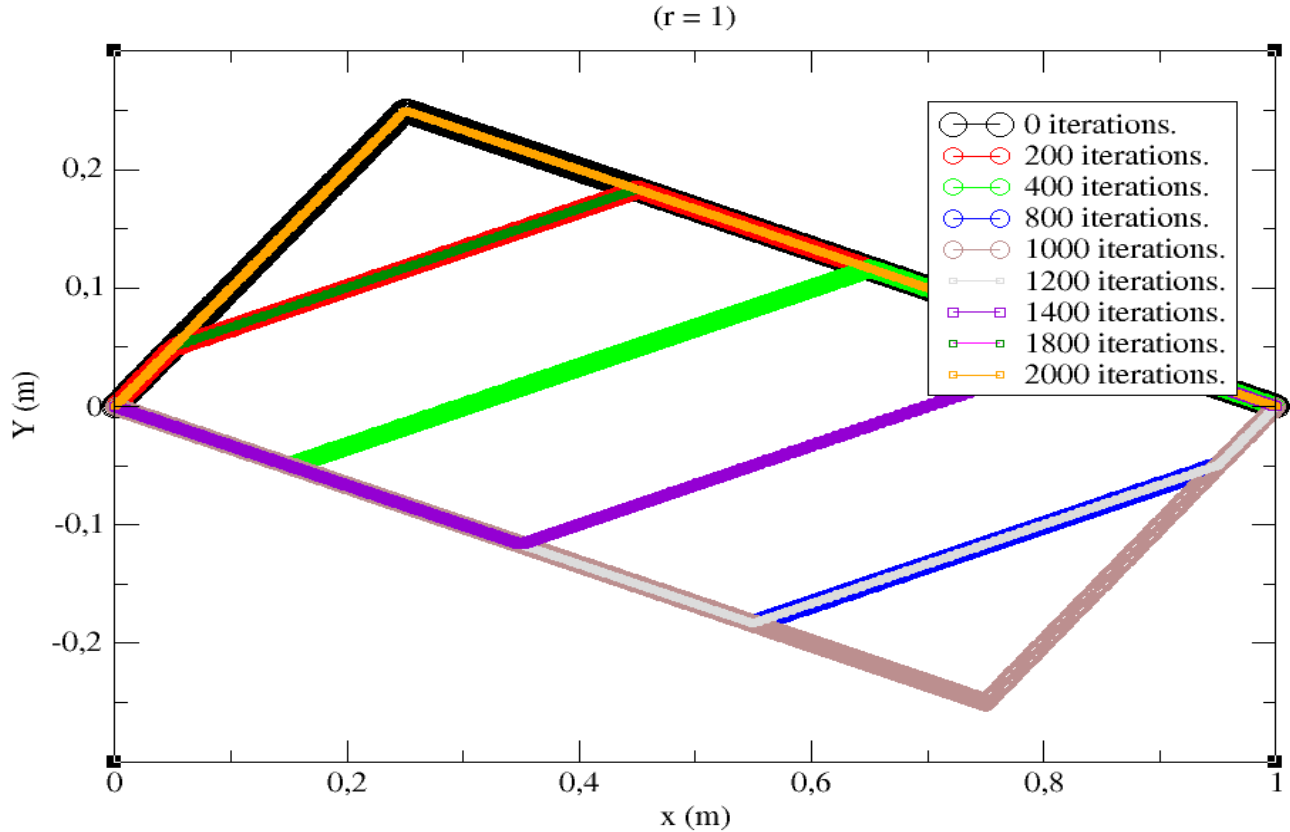


Figura 8: Deslocamento de uma onda unidimensional. A simulação começa na curva preta (em cima).

Tarefa 3

(III) Considere o item (I) mas com $Y_0(x)$ dado como num violão. Consideremos o programa do item anterior com as extremidades fixas. Rode-o para um pacote Gaussiano inicial localizado em $x_0 = L/2$. Armazene os resultados de $Y(L/4, t)$ para vários tempos. Façamos uma análise espectral dos dados calculando-se as transformadas de Fourier temporais senos e cossenos dos dados (use o seu programa desenvolvido no primeiro projeto). A maneira mais rápida de extrairmos boa parte da física do problema é analisarmos, ao invés das transformadas senos $Y_s(f)$ ou cossenos $Y_c(f)$ o espectro de potências $P(f) = (Y_s(f))^2 + (Y_c(f))^2$. Mostre o gráfico $P(f) \times f$.

(IIIa) Compare os picos obtidos com as frequências correspondentes aos modos estacionários. Os comprimentos de onda de tais modos são: $\lambda_1 = \frac{2L}{1}$, $\lambda_2 = \frac{2L}{2}$, $\lambda_3 = \frac{2L}{3}$, ..., $\lambda_k = \frac{2L}{k}$, $k = 1, 2, \dots$. Veja a figura abaixo.

Consequentemente as frequências normais são $f_k = \frac{c}{\lambda_k} = \frac{ck}{2L}$, $k = 1, 2, \dots$. Apareceram que modos? Faltam alguns?

Considere nos itens (b)-(e) abaixo pacotes gaussianos com centro em x_0 e meia largura $\sigma = L/30$.

(IIIb) Inicie o pacote Gaussiano na posição $x_0 = \frac{L}{4}$ e repita a análise espectral anterior. Que modos aparecem agora? Quais estão perdidos?

(IIIc) Inicie o pacote Gaussiano na posição $x_0 = \frac{L}{3}$ e repita a análise anterior. Que modos surgem?

(IIId) Inicie agora com o pacote em $x_0 = \frac{L}{20}$ e repita a análise. Que modos faltam? Como entender tais resultados?

Agora que você já "conversou com os Deuses" e tem a sua teoria vamos testá-la.

Para essa tarefa, novamente, o programa principal fica praticamente inalterado e apenas modifiquei o módulo a ser importado ("wave_wfe_guitar_FT") pra inicializar a onda com o pulso de violão e calcular a transformada de Fourier discreta em um ponto especificado no espaço ². Vide a figura 9.

```
74 |
75 | ! simulates waves until time t, saving Y(t) at x = x_0 (e.g., L/4)
76 | subroutine simulate_and_getFT(self, t, x_0, filename)
77 |   class(wave), intent(in out) :: self
78 |   integer, intent(in)          :: t                ! number of iterations.
79 |   real(dp), intent(in)         :: x_0              ! position to get signal.
80 |   character(len=*), intent(in) :: filename         ! file to overwrite data.
81 |   complex(dpc), allocatable    :: data(:), FTdata(:) ! signal and its FT.
82 |   integer                      :: i, io, x0_idx
83 |   real(dp)                     :: dt
84 |
85 |   allocate(data(t+1))
86 |   allocate(FTdata(t+1))
87 |   x0_idx = int(self%npoints * x_0) ! index of x_0 to use in self%sys(:, :)
88 |   dt      = self%dx*self%r/self%c
89 |
90 |   !
91 |   do i = 1, (t + 1)
92 |     data(i) = cmplx(self%sys(x0_idx, 2), kind=dp)
93 |     call self%iterate()
94 |   end do
95 |
96 |   call FT(data(:), FTdata(:), t+1)
97 |
98 |   open(newunit=io, file=filename, action="write")
99 |   do i = 1, t+1
100 |     write(io, *) (i-1)/(dt*real(t+1, dp)), abs(FTdata(i))
101 |   end do
102 |   close(io)
103 |
104 | end subroutine simulate_and_getFT
105 |
106 | end module wave_wfe_guitar_FT
```

Figura 9: Mudança na subrotina de simular: calcula transformada de Fourier de um ponto especificado também.

O espectro de potências da onda de violão (com pico inicial e ponto de medição da transformada de Fourier em $x = L/4$) está dada na figura 10 (**como o sinal é real, só será mostrado a primeira metade dos espectros de potências para os gráficos a seguir**). Vale a pena notar que para ter uma boa quantia de dados para a transformada, tive que iterar a simulação para 10 mil passos. Dessa forma, os picos no espectro de potências ficarão mais definidos.

²Isso foi feito importando o módulo criado no projeto 1.

Power spectrum of guitar string.

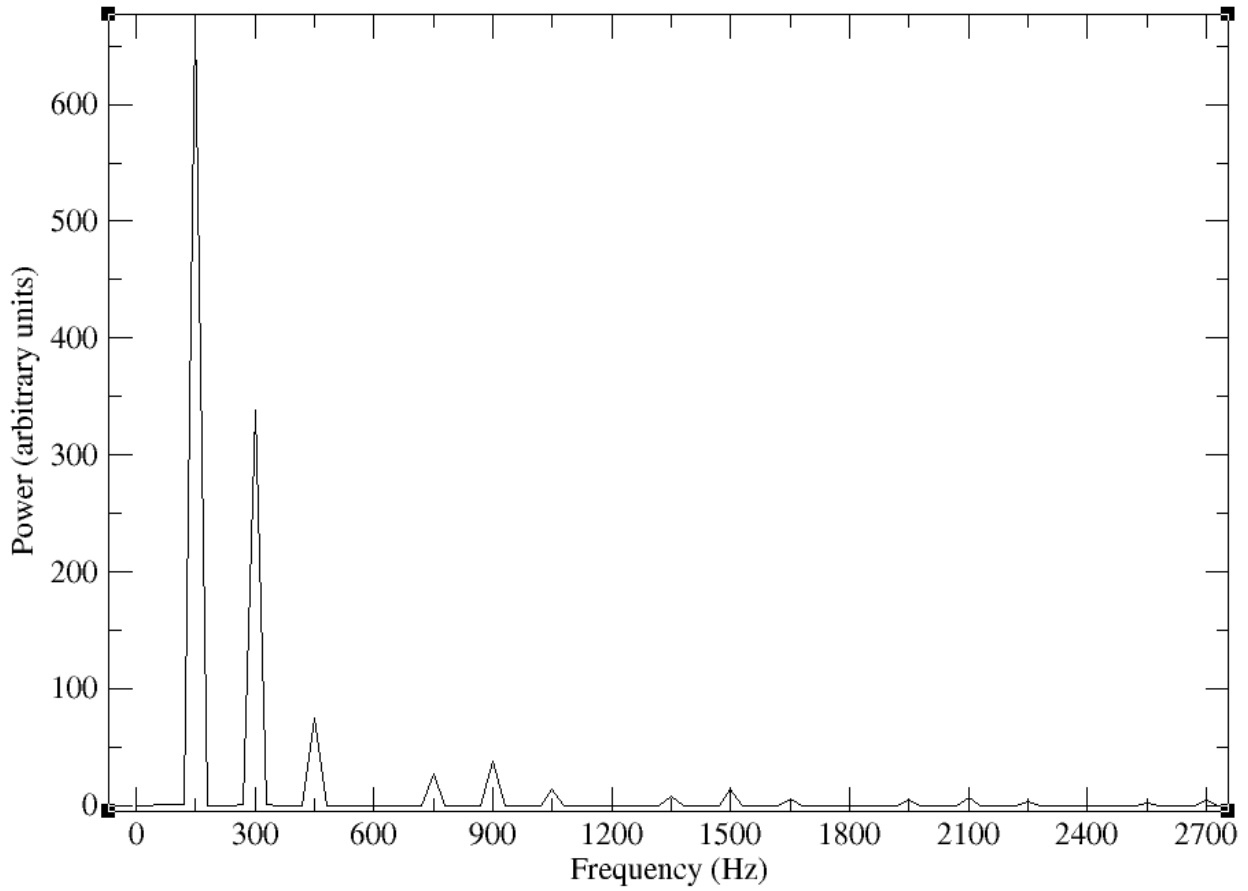


Figura 10: Espectro de potências para onda de violão. O pico inicial e a posição de medição da transformada de Fourier foram em $x = L/4$.

As frequências correspondentes aos modos normais são dados pela equação 3. Para os parâmetros usados nesse projeto ($L = 1m$ e $c = 300m/s$), essas frequências serão: $150Hz$, $300Hz$, etc. Pela figura 10, pode ser notado que faltam apenas as frequências: $600Hz$, $1200Hz$, etc.

$$f_k = \frac{c}{\lambda_k} = \frac{ck}{2L} \quad , \quad k = 1, 2, 3, \dots \quad (3)$$

Nos gráficos 11, 12 e 13, estão os espectros de potências para pulsos iniciais gaussianos com meia largura igual a $\sigma = L/30$ e posições iniciais dados por x_0 (especificados em cada gráfico). Pode ser notado que as frequências sempre aparecem em múltiplos da frequência $150Hz$. Para $x_0 = L/4$, faltam as frequências: $600Hz$, $1200Hz$ etc. Para $x_0 = L/3$, faltam as frequências: $450Hz$, $600Hz$, $900Hz$ etc. Para $x_0 = L/20$, faltam as frequências: $600Hz$, $1200Hz$ etc.

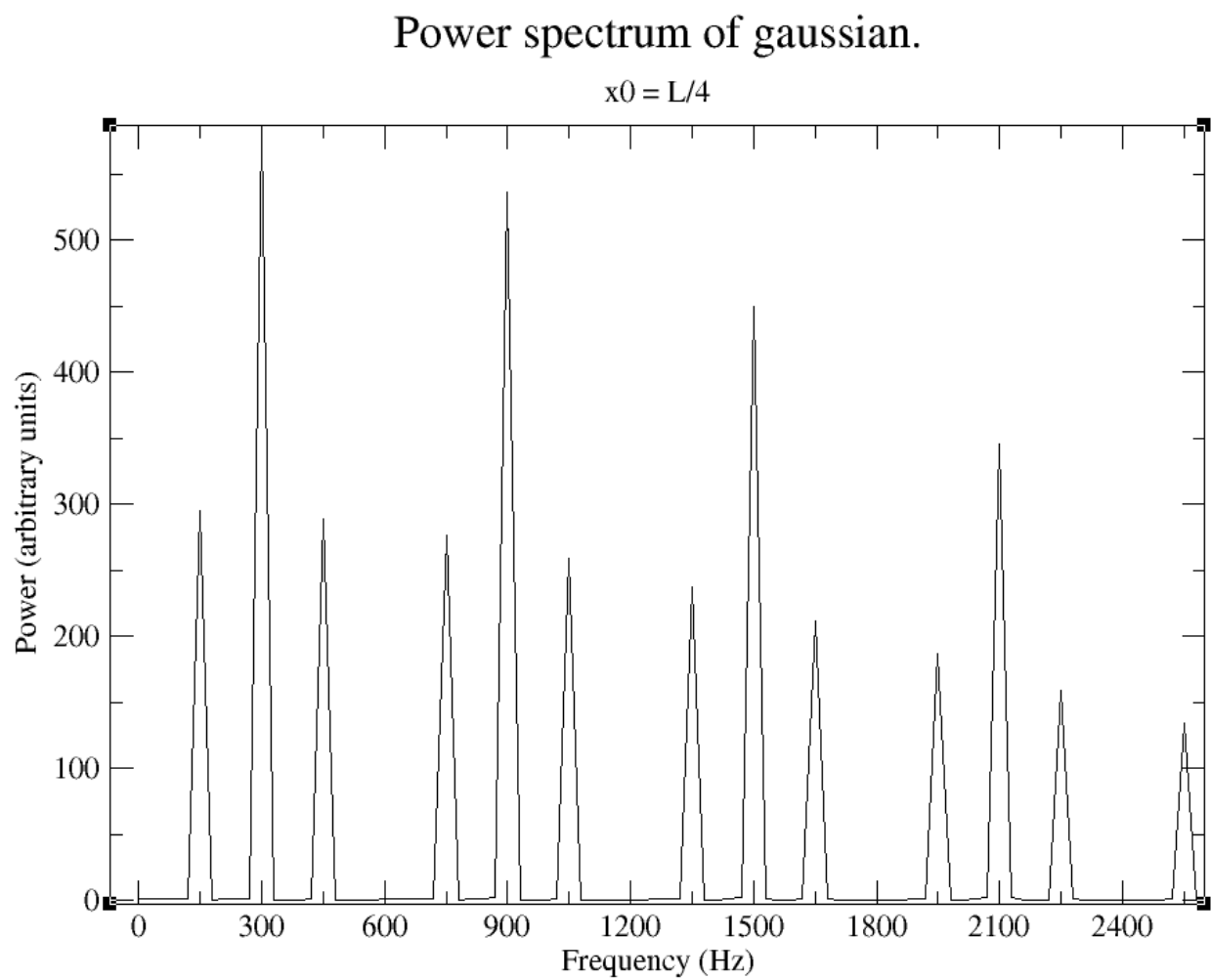


Figura 11: Espectro de potências para onda gaussiana. O pico inicial e a posição de medição da transformada de Fourier foram em $x = L/4$.

Power spectrum of gaussian.

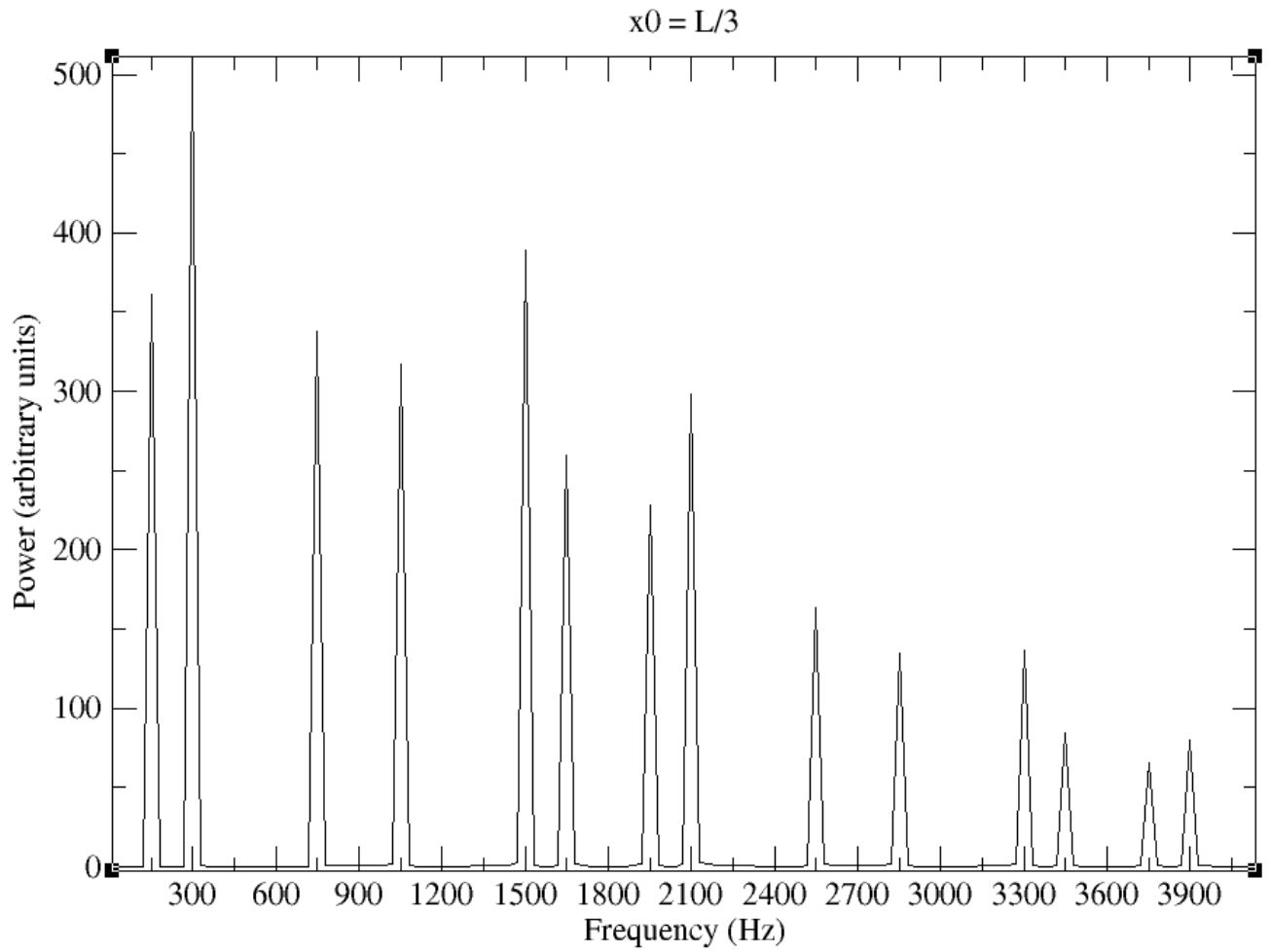


Figura 12: Espectro de potências para onda gaussiana. O pico inicial foi em $x = L/3$ e a posição de medição da transformada de Fourier foi em $x = L/4$.

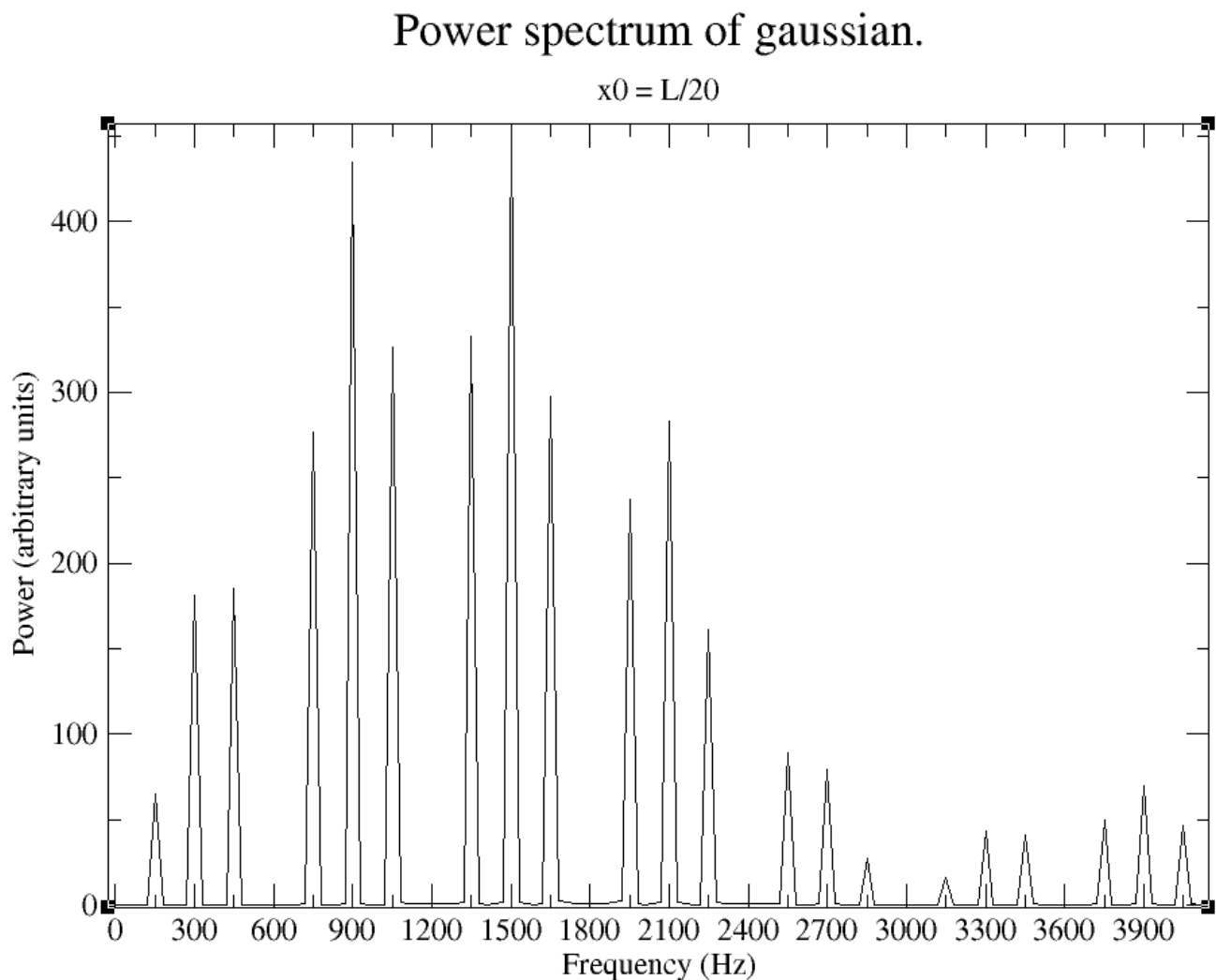


Figura 13: Espectro de potências para onda gaussiana. O pico inicial foi em $x = L/20$ e a posição de medição da transformada de Fourier foi em $x = L/4$.

A explicação para os gráficos 11, 12, 13 é a seguinte: a onda inicial começa com um certo formato (gaussiana ou violão) que pode ser entendido como uma superposição dos modos estacionários da corda, cada uma com uma certa frequência. A onda se propaga devido à vibração dos modos estacionários (um modo abaixa enquanto outro levanta, tal que a soma dá a impressão de uma onda se propagando). Como a onda é perfeita (não ocorre dispersão da onda ou dissipação de energia), é possível assumir que essa combinação dos modos normais fica a mesma com o passar do tempo.

Ademais, nos gráficos 10, 11, 12 e 13, o maior pico no espectro de potências corresponderá ao modo estacionário que melhor representa a onda inicial. O k -ésimo modo estacionário tem k picos e $k - 1$ nós, com o primeiro pico pra cima a uma distância $L/(2k)$ da origem e o primeiro nó a uma distância L/k da origem.

Logo, no gráfico 10 (violão), a onda inicial mais se aproxima do primeiro modo porque o pico do primeiro modo no espectro de potências é grande.

No gráfico 11, a onda inicial tem pico no mesmo lugar que o modo estacionário com $k = 2$ ($f = 300\text{Hz}$), então esse será um pico proeminente, enquanto os modos com $k = 4$ ($f = 600\text{Hz}$), $k = 8$ ($f = 1200\text{Hz}$), ... tem nós nesse pico, então estes serão coibidos.

No gráfico 12, a onda inicial tem pico entre os modos estacionários com $k = 1$ ($f = 150\text{Hz}$) e

$k = 2$ ($f = 300Hz$), então esses serão proeminentes, enquanto os modos com $k = 3$ ($f = 450Hz$), $k = 6$ ($f = 900Hz$), ... tem nós exatamente nesse pico, então estes serão coibidos.

No gráfico 13, a onda inicial tem pico no modo estacionário em $k = 10$ ($f = 1500Hz$), então esses serão proeminentes, enquanto o modo com $k = 20$ ($f = 3000Hz$) tem nó exatamente nesse pico, então este será coibido.

Tarefa 4

(IIIe) Refaça os programas do projeto 2 para o caso em que uma das extremidades, por exemplo $x = 0$, é fixa e a outra $x = L$ é livre. Teste suas conclusões escolhendo apropriadamente a posição inicial x_0 para alguns pacotes iniciais. A condição de borda livre é aquela em que a derivada espacial parcial da amplitude é nula. Isto implica que a amplitude na borda é a mesma que a do ponto adjacente.

(Decidi separar essa questão porque é grande)

A condição de uma extremidade livre em $x = L$ foi implementada impondo que a cada iteração (e uma vez na inicialização), a amplitude naquela extremidade e do ponto adjacente coincidem, pois:

$$\frac{dY(x,t)}{dx} = 0 \implies \frac{Y_{i+1} - Y_i}{\Delta x} = 0 \implies Y_{i+1} = Y_i$$

Não vou incluir o trecho do código que implementa isso porque é trivial. O restante do código se mantém inalterado e idêntico aos códigos anteriores ³.

Nos gráficos a seguir, a condição da extremidade livre só fez com que a onda que chega nessa extremidade é refletida do mesmo lado que incidiu (não será refletida no sentido oposto que nem para extremidades fixas).

Para testar o código com essa condição de contorno, repeti a onda inicial da tarefa 1 (gaussiana em $x = L/3$). Na figura 14 está esse pulso propagando na corda com o tempo. Eu escolhi $r = 2$ e $r = 0.25$ também, mas obtive os mesmos resultados que as simulações com extremidades fixas ($r = 2$ há divergência e $r = 0.25$ é praticamente igual a $r = 1$), então vou omitir os gráficos. Além disso, o tempo que demora para repetir a condição inicial é a mesma também: $t = 2L/rdx = 2000$ iterações (aproximadamente $6.7ms$. Novamente escolhi os mesmos parâmetros: $\Delta x = 10^{-3}m$ e $r = 1$).

³Eu incluí os módulos tudo no programa principal junto com algumas linhas específicas para resolver tarefas diferentes. Porém, decidi comentar tudo que eu não ia precisar, porque há nomes em comum nas subrotinas dos módulos. Por exemplo, se eu quisesse simular a corda de violão, eu incluiria só o módulo "wave_fre_guitar" apenas e chamaria "initialize" e "simulate".

Propagations of 1D wave with free end.

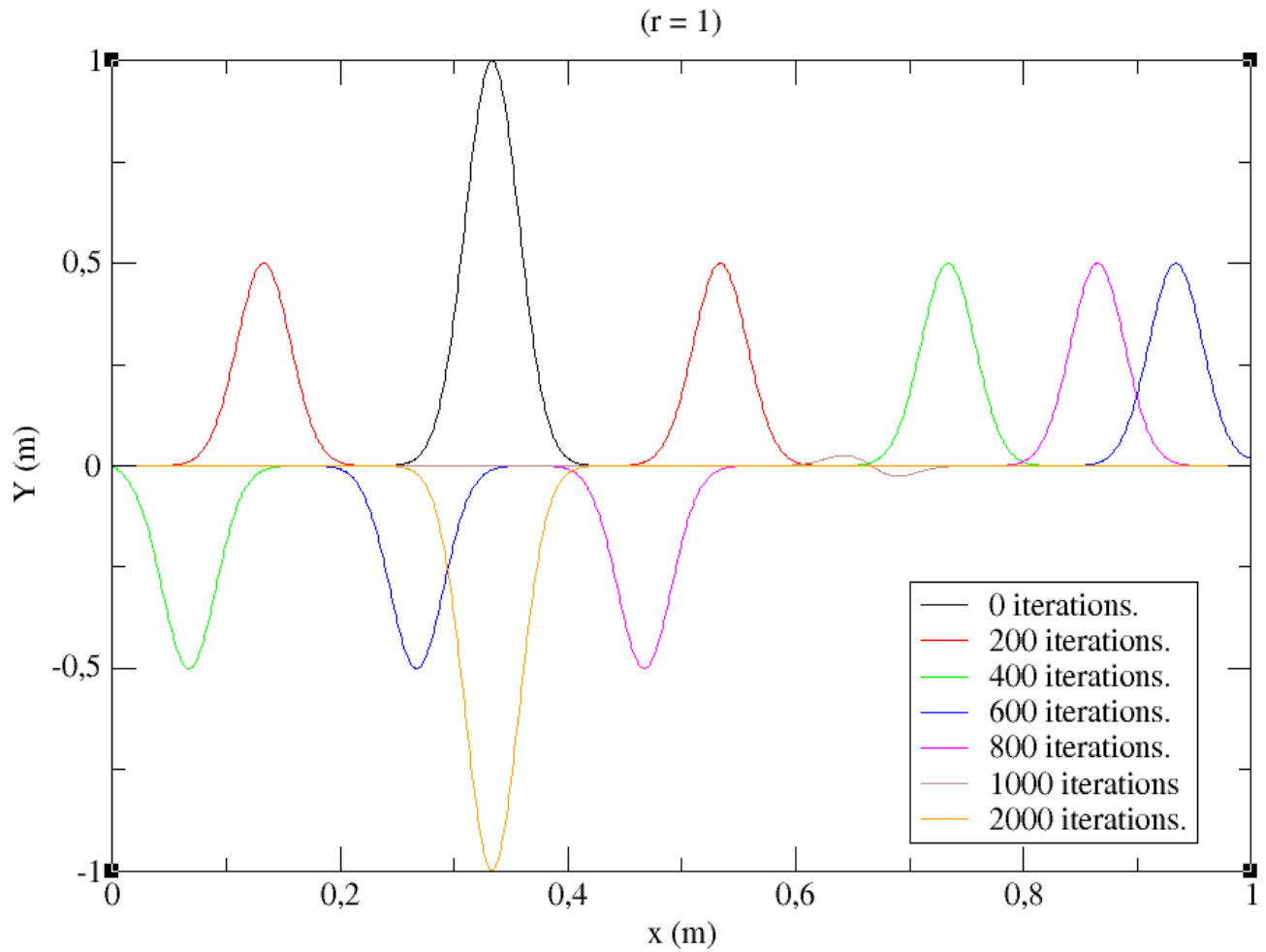


Figura 14: Deslocamento de uma onda unidimensional com a extremidade direita livre. A onda começa na configuração em preto.

Para a tarefa 2 (violão em $x = L/4$), obtive o resultado nas figuras 15 e 16 (dividi o gráfico em dois porque o movimento é um pouco mais complicado). De novo, a onda que bate na extremidade fixa (esquerda) volta com amplitude invertida, enquanto a onda que bate na extremidade livre (direita) volta do mesmo lado. O tempo para repetir a condição inicial é a mesma: $t = 2000$ iterações (aproximadamente $6.7ms$).

Propagation of 1D wave with free end.

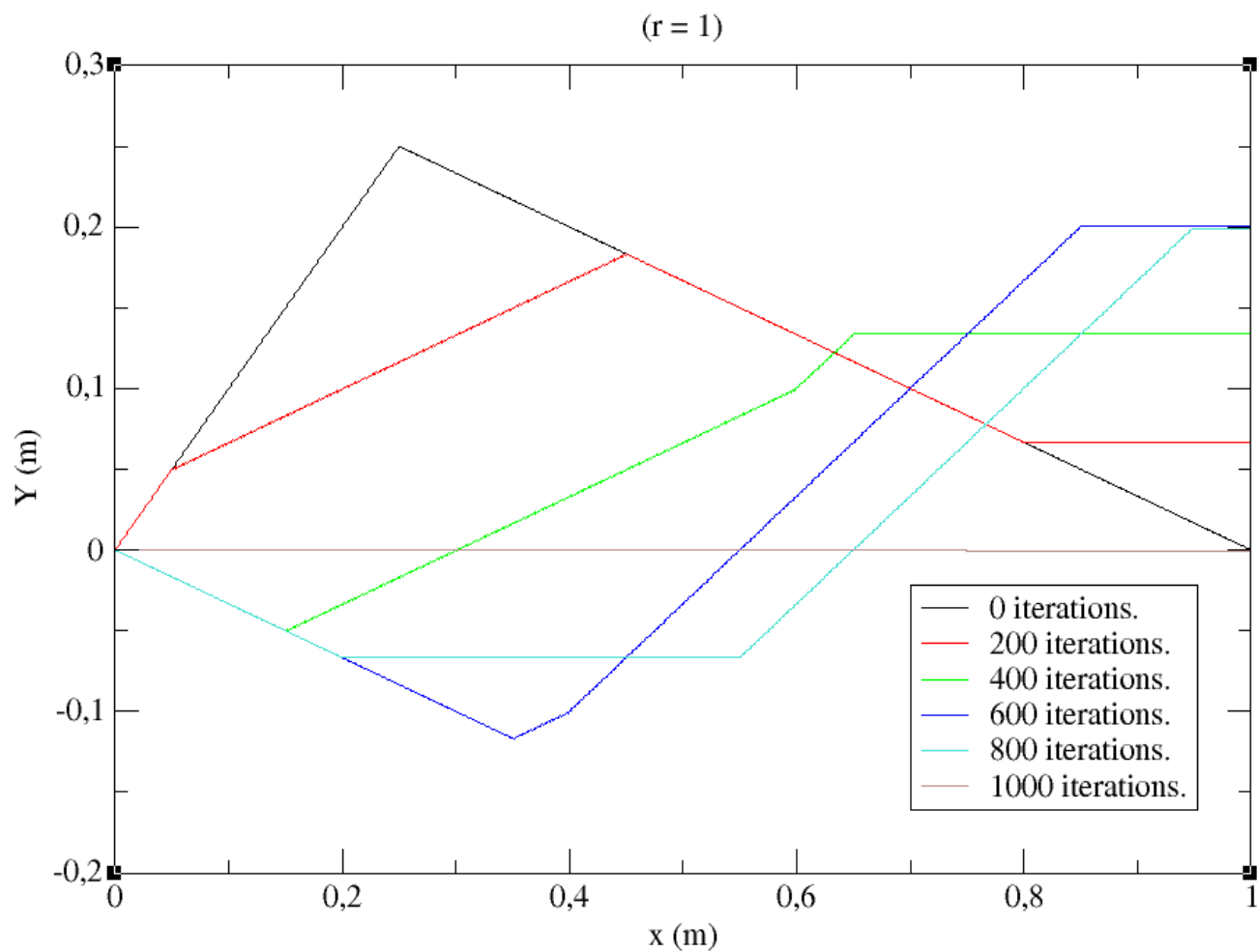


Figura 15: Deslocamento de uma onda unidimensional com a extremidade direita livre. A onda começa na configuração em preto. Parte 1.

Propagation of 1D wave with free end.

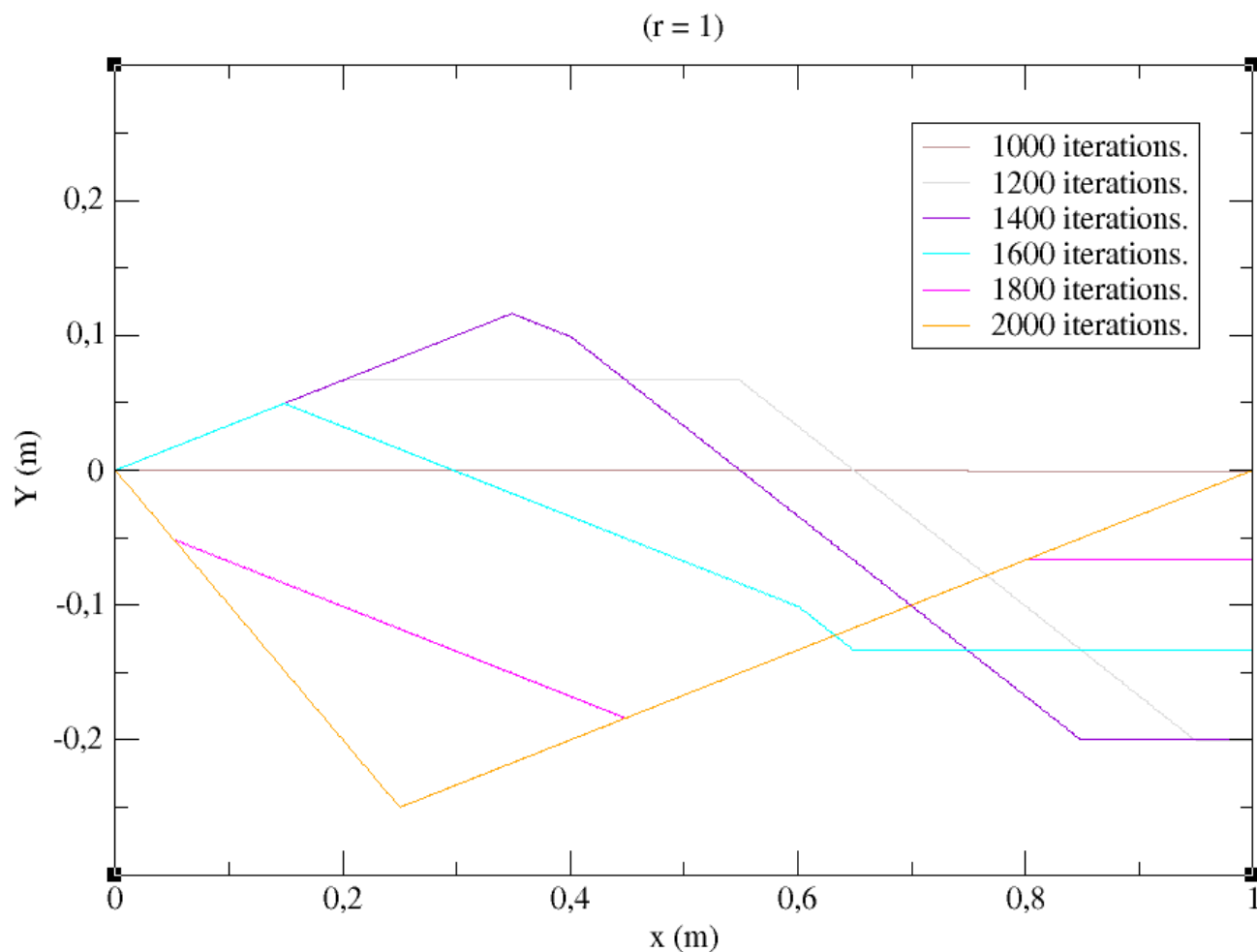


Figura 16: Deslocamento de uma onda unidimensional com a extremidade direita livre. A onda começa na configuração em preto. Parte 2.

Para a tarefa 3, os resultados estão nos gráficos 18 - 21. Novamente, o código fica praticamente inalterado com o código das tarefas 3a-3d, sendo a única exceção a imposição da condição de contorno.

Para entender esses resultados, precisamos recorrer aos modos estacionários de uma onda com uma extremidade livre e a outra presa (vide figura 17). Os comprimentos de onda e frequências possíveis desses modos estacionários serão dados pela equação 4.

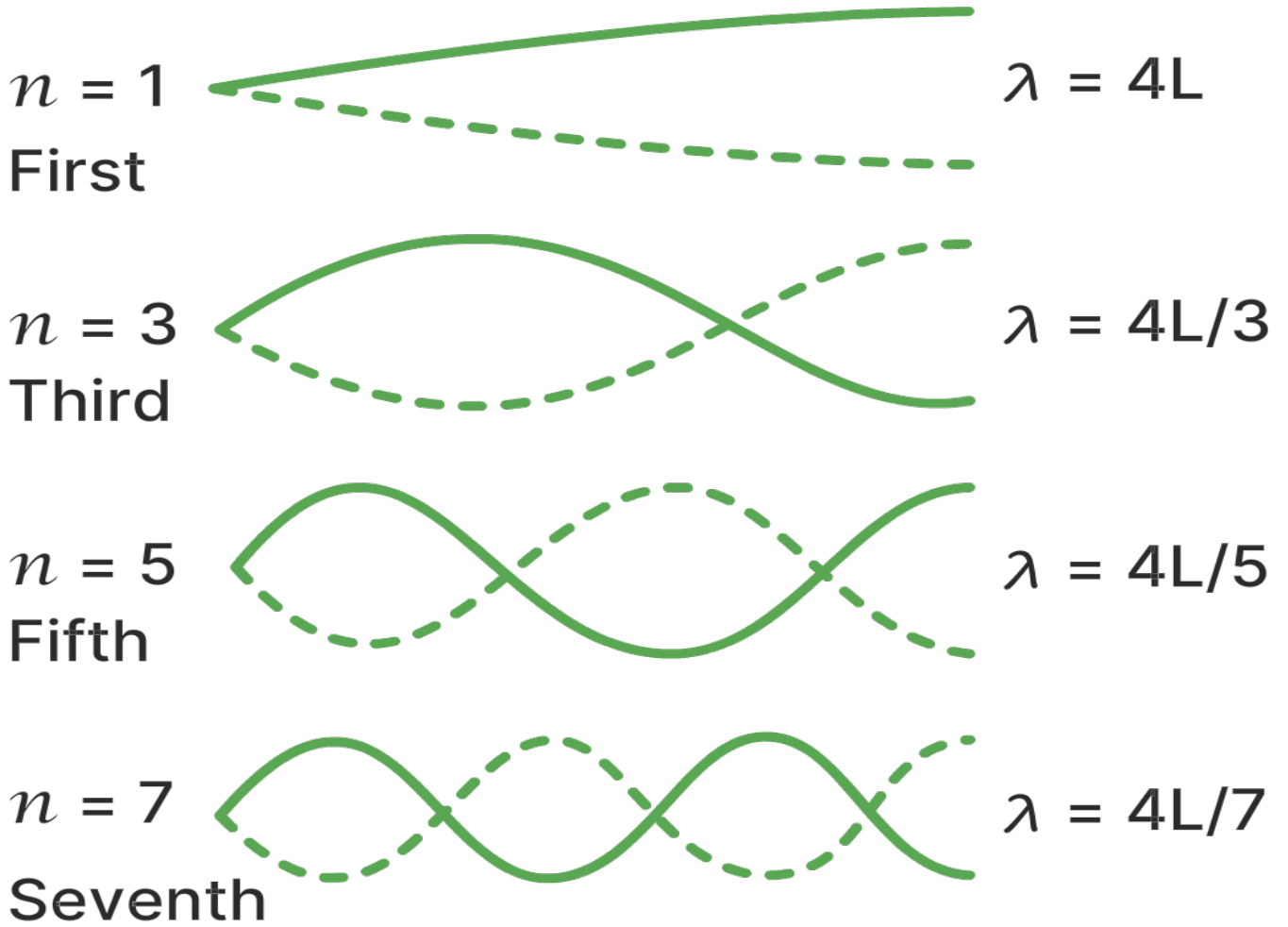


Figura 17: Modos estacionários de uma onda com uma extremidade fixa e outra livre. Fonte: olabs.edu.in.

$$\lambda_k = \frac{4L}{2k-1} \iff f_k = \frac{c(2k-1)}{4L} \quad , \quad k = 1, 2, 3, \dots \quad (4)$$

Para $c = 300m/s$ e $L = 1m$, essas frequências serão: $f_k = 75Hz, 225Hz, 375Hz, \dots$

Então, analogamente às tarefas 3a-3d, os picos nos gráficos 18 - 21 corresponderão aos modos estacionários que mais se assemelham ao formato de onda inicial. A localização dos picos e nós dos modos estacionários serão dados pelas fórmulas 5 e 6.

$$p_k = \frac{L}{2k-1} \quad , \quad k = 1, 2, 3, \dots \quad (5)$$

$$n_k = \frac{2L}{2k-1} \quad , \quad k = 1, 2, 3, \dots \quad (6)$$

No gráfico 18, a onda de violão com pico inicial em $x = L/4$ se assemelha mais com a onda estacionária com pico em $x = L/3$ (que tem frequência $225Hz$, i.e., o maior pico no espectro de potências).

No gráfico 19, a onda gaussiana com pico inicial em $x = L/4$ se assemelha mais com as ondas estacionárias com picos em $x = L/3$ e $x = L/5$ (que têm frequências $225Hz$ e $375Hz$, i.e., os maiores picos no espectro de potências).

No gráfico 20, a onda gaussiana com pico inicial em $x = L/3$ se assemelha mais com a onda estacionária com pico em $x = L/3$ (que tem frequência $225Hz$, i.e., o maior pico no espectro de potências).

No gráfico 21, a onda gaussiana com pico inicial em $x = L/20$ se assemelha mais com as ondas estacionárias com picos em $x = L/19$ e $x = L/21$ (que têm frequências $1425Hz$ e $1575Hz$, i.e., os maiores picos no espectro de potências).

Power spectrum of guitar string with a free end.

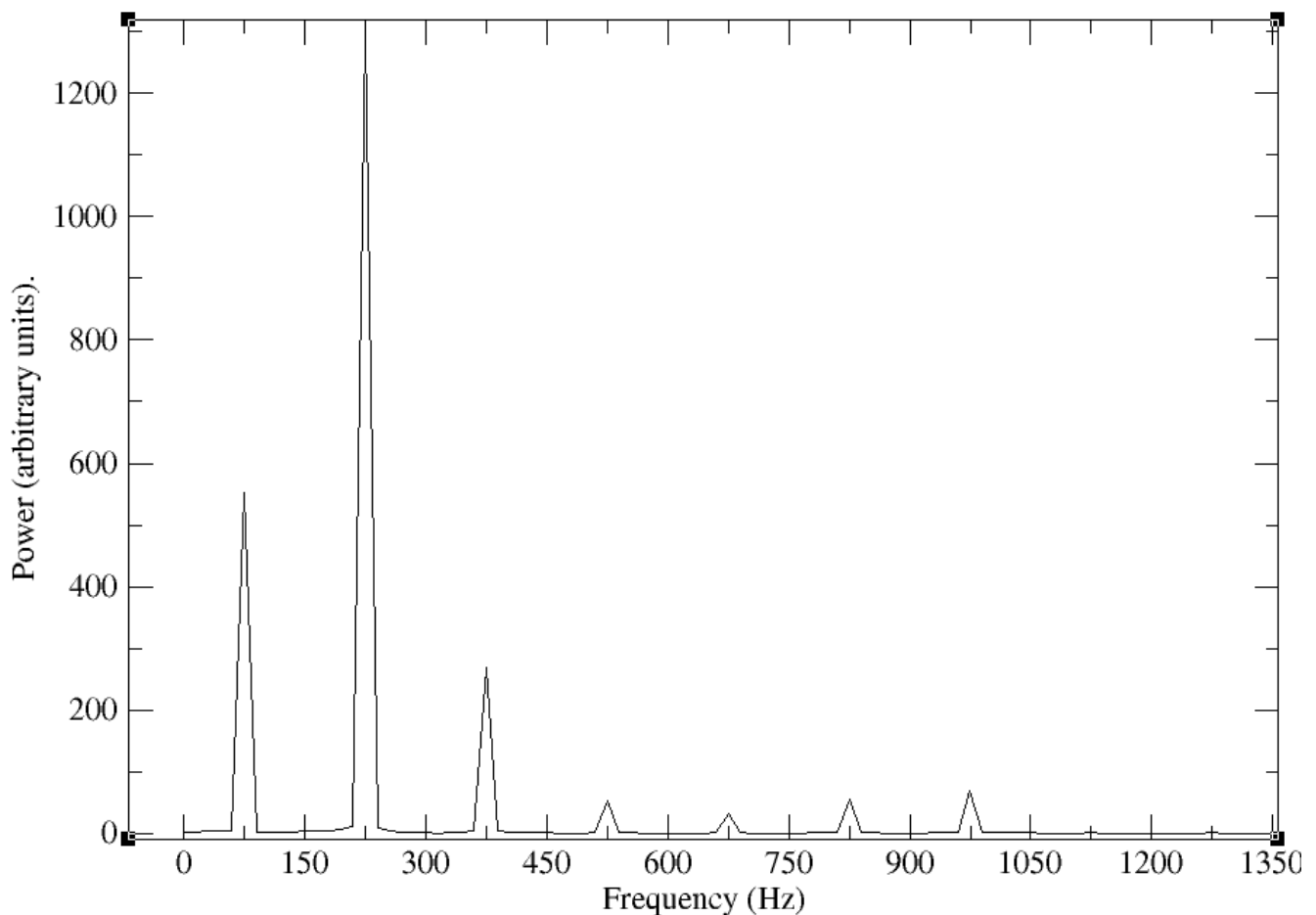


Figura 18: Espectro de potências para onda de violão com uma extremidade livre. O pico inicial foi em $x = L/4$ e a posição de medição da transformada de Fourier foi em $x = L/4$.

Power spectrum of gaussian with one free end.

($x_0 = L/4$)

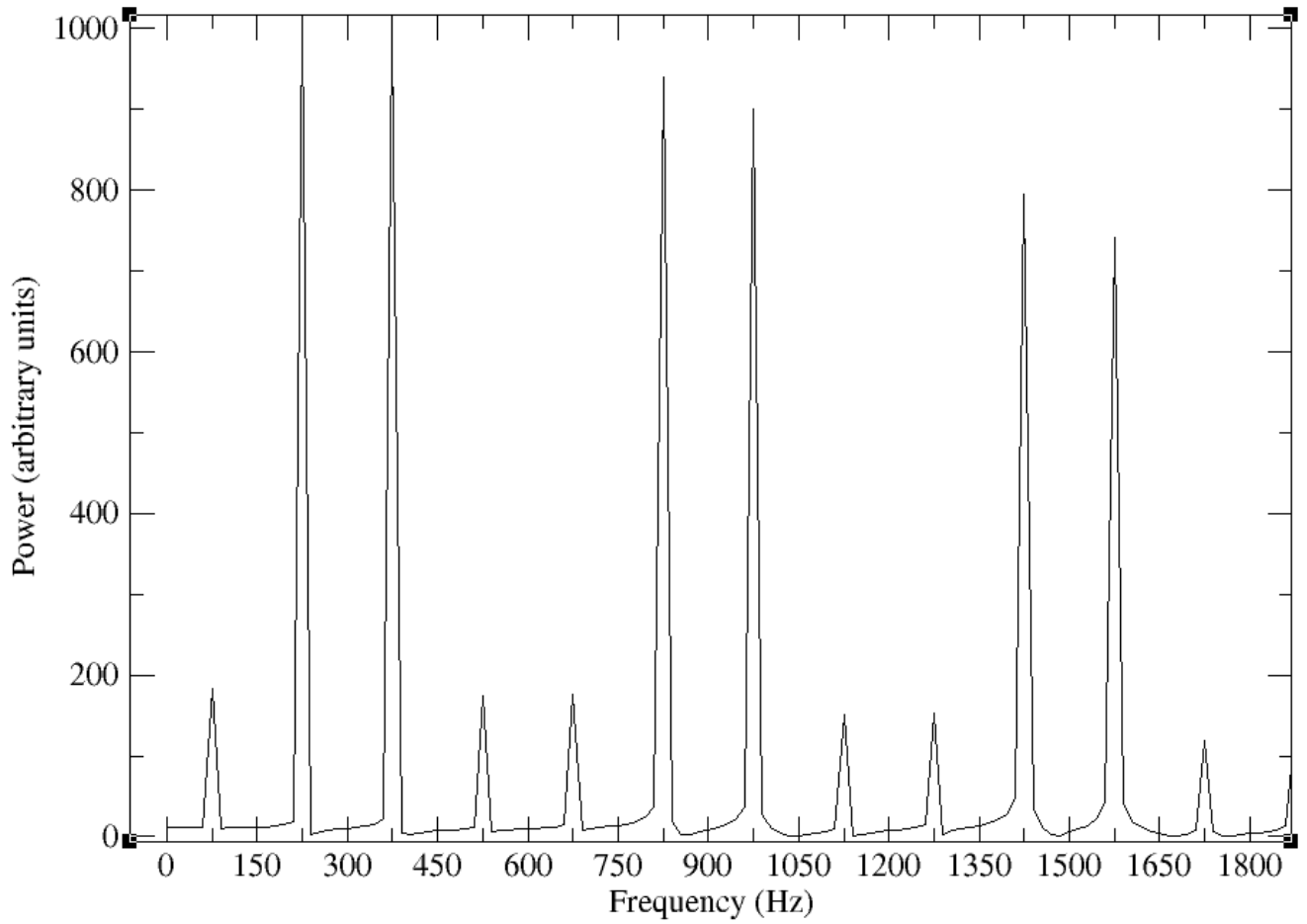


Figura 19: Espectro de potências para onda gaussiana com uma extremidade livre. O pico inicial foi em $x = L/4$ e a posição de medição da transformada de Fourier foi em $x = L/4$.

Power spectrum of gaussian with one free end.

($x_0 = L/3$)

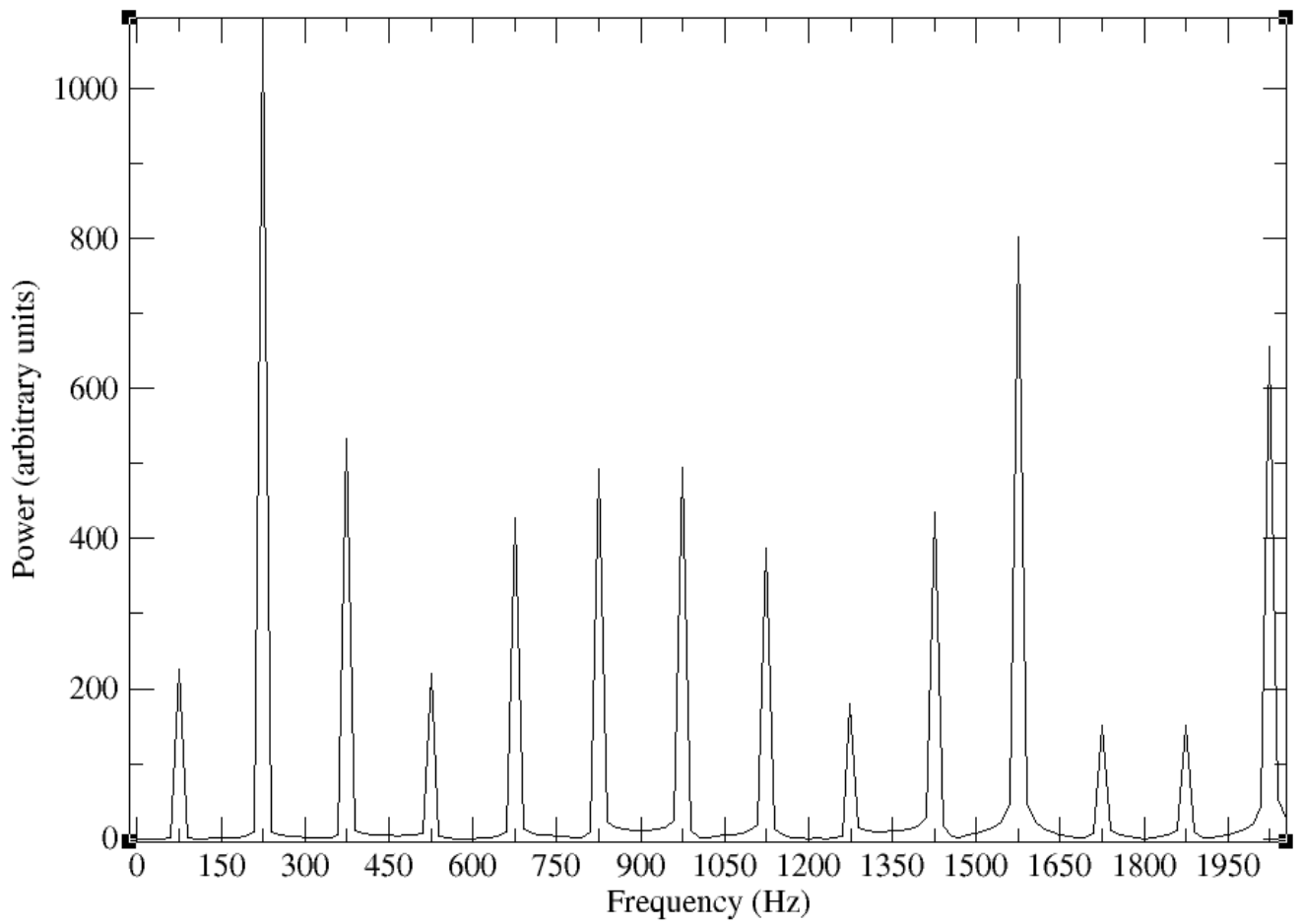


Figura 20: Espectro de potências para onda gaussiana com uma extremidade livre. O pico inicial foi em $x = L/3$ e a posição de medição da transformada de Fourier foi em $x = L/4$.

Power spectrum of gaussian with one free end.

($x_0 = L/20$)

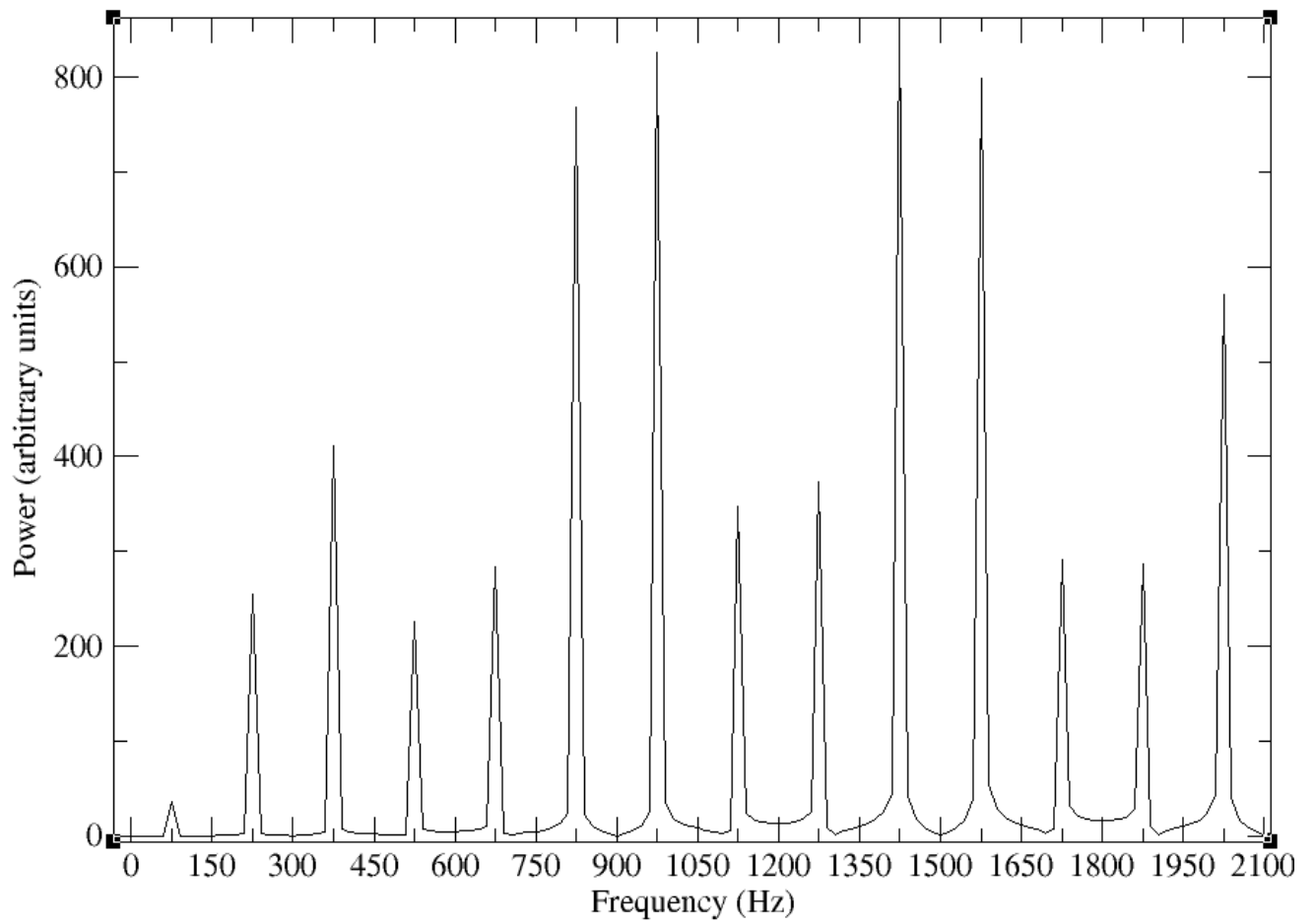


Figura 21: Espectro de potências para onda gaussiana com uma extremidade livre. O pico inicial foi em $x = L/20$ e a posição de medição da transformada de Fourier foi em $x = L/4$.